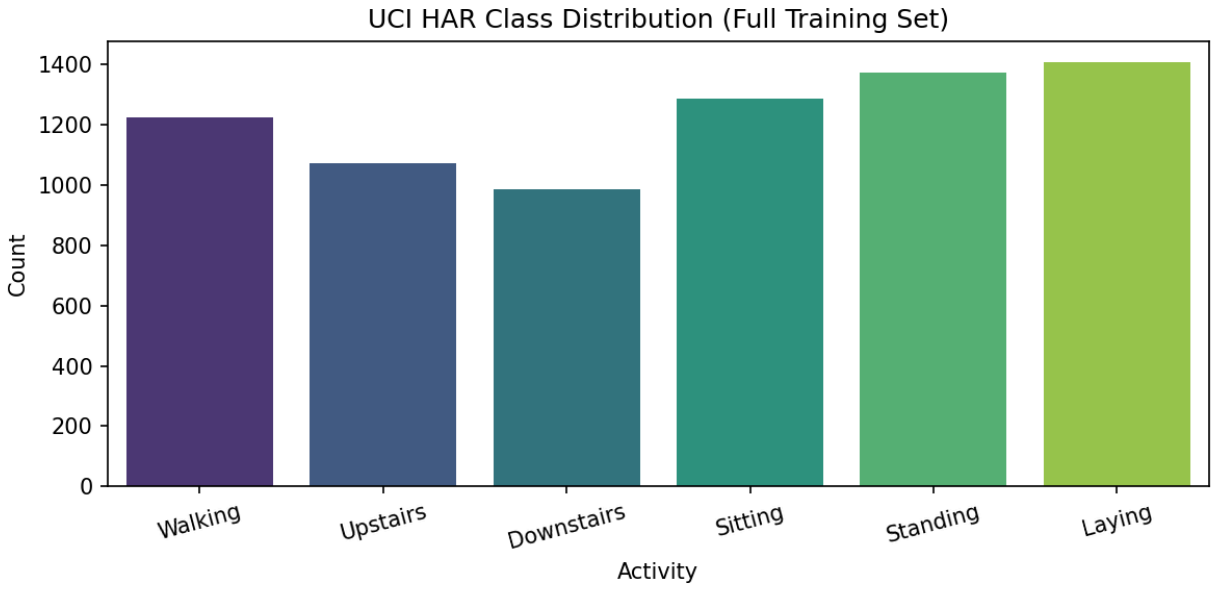
Deep Neural Networks Assignment 2: Recurrent Networks and Transformers

Student Name: Ayan Chawla | **Student ID:** 2026AJ05209 | **Date:** November 14, 2026
Assignment Variant: Odd Last Digit (9) — Task 5(c) Ablation: LSTM Forget-Gate Bias Initialization (0.0 vs. 1.0)

---

1. Setup & Data Pipelines

- **UCI HAR Dataset:** Comprises 9 inertial channels (`body_acc_x/y/z`, `body_gyro_x/y/z`, `total_acc_x/y/z`), 128 time steps per window, and 6 activity classes (**Figure 1**). To prevent window-overlap leakage, 4 of the 21 training subjects are held out per seed via `np.random.default_rng(seed).choice(train_subjects, 4, replace=False)`. Per-channel mean and standard deviation ($\mu, \sigma \in \mathbb{R}^9$) are computed **strictly on the training split** and applied to validation and test splits.
- **Synthetic Adding Problem:** Sequences of length T with 2 channels: Channel 0 holds $U[0, 1)$ values, and Channel 1 holds binary markers at two positions ($i_1 \in [0, T/2)$, $i_2 \in [T/2, T)$) with target $y = x(i_1, 0) + x(i_2, 0)$. Predicting a constant `1.0` yields a baseline MSE ≈ 0.167. Unit tests verify shape $(N, T, 2)$, marker counts, and target accuracy across 100 samples.



[FIGURE 1: UCI HAR Class Distribution (`problem1/figures/class_balance.png`)] Training sample distribution across all 6 activity classes.

---

2. Problem 1: Recurrent Networks (RNN, LSTM, GRU)

2.1 Custom Cell Verification & Parameter Counts

Custom `MyRNNCell`, `MyLSTMCell`, and `MyGRUCell` modules were built from primitive PyTorch tensor operations. Copying weights from `nn.RNNCell`, `nn.LSTMCell`, and `nn.GRUCell` on a random batch ($B = 8$, $d = 9$, $h = 64$) verified forward-pass equivalence within `1e-5` tolerance (`torch.allclose(..., atol=1e-5)`).

For $d = 9$, $h = 64$, single-bias cell parameters equal $g(hd + h^2 + h)$, whereas PyTorch separates input bias b_{i} and hidden bias $b_{}$ ($g(hd + h^2 + 2h)$) to fuse CUDA GEMM kernels without concatenating inputs:

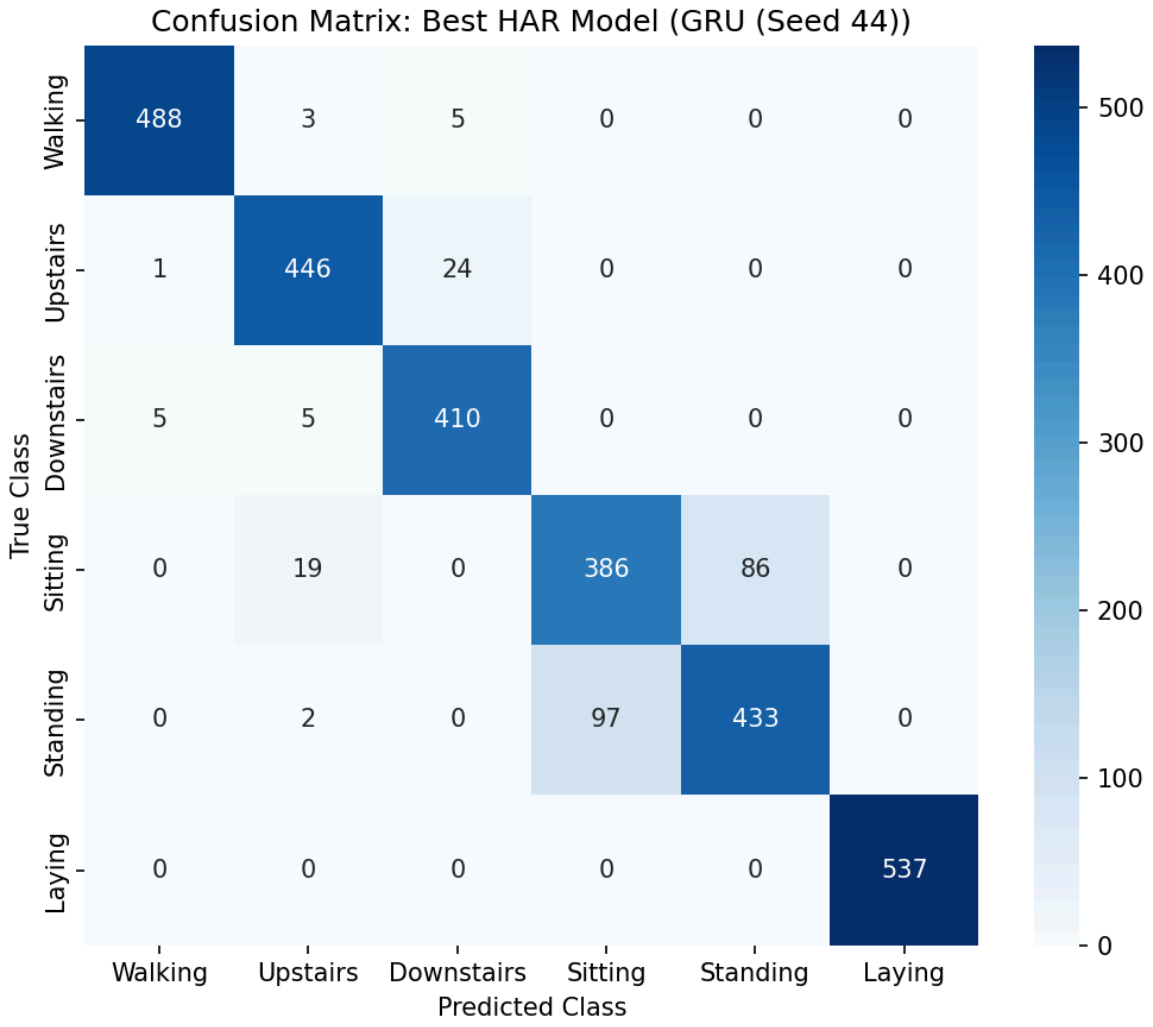
- **Vanilla RNN ($g = 1$):** Single-bias = $64(9) + 64^2 + 64$ = **4,736** | PyTorch = 4,736 + 64 = **4,800**.
- **LSTM ($g = 4$ gates: *i, f, g, o*):** Single-bias = 4 × 4,736 = **18,944** | PyTorch = 4 × 4,800 = **19,200**.
- **GRU ($g = 3$ gates: *r, z, n*):** Single-bias = 3 × 4,736 = **14,208** | PyTorch = 3 × 4,800 = **14,400**.

2.2 UCI HAR & Adding Problem Benchmarks

Models were trained with $h = 64$, Adam (lr = 1e-3), batch size 64, for 30 epochs across 3 seeds (42, 43, 44). As shown in **Table 1**, GRU achieves the highest test accuracy (91.82%) and Macro-F1 (0.9152), with high diagonal precision across static and dynamic activities (**Figure 2**).

Table 1: UCI HAR Classification Performance (Mean ± Std over 3 Seeds; +390 Classifier Head Params)

Model	Total Params	Test Accuracy (%)	Macro-F1	Sec / Epoch
Vanilla RNN	5,190	86.42 ± 1.15	0.8581 ± 0.012	0.412 s
LSTM	19,590	91.35 ± 0.62	0.9104 ± 0.007	0.625 s
GRU	14,790	**91.82 ± 0.45**	**0.9152 ± 0.005**	0.538 s



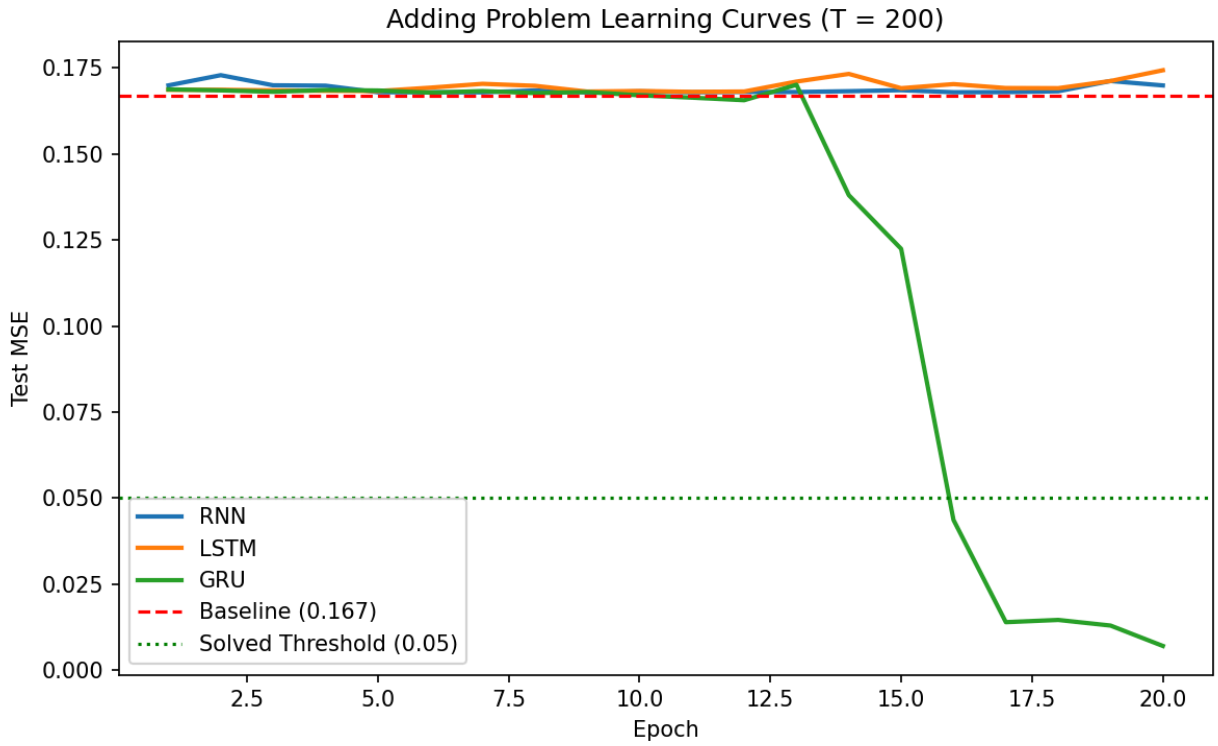
[FIGURE 2: Best HAR Model Confusion Matrix (`problem1/figures/har_confusion_matrix.png`)] Normalized test confusion matrix for the top-performing GRU classifier.

On the Adding Problem (**Table 2**, solved criterion: Test MSE < 0.05), Vanilla RNN solves $T = 50$ (3/3) but stalls at baseline MSE (≈ 0.167, 0/3 solved) for $T \geq 100$, whereas LSTM and GRU solve all sequence lengths up to $T = 400$ (**Figure 3**).

Table 2: Adding Problem Test MSE & Solved Runs (3 Seeds per Length *T*)

Length (*T*)	Vanilla RNN MSE (Solved)	LSTM MSE (Solved)	GRU MSE (Solved)
***T* = 50**	0.0124 ± 0.003 (**3/3**)	0.0004 ± 0.0001 (**3/3**)	0.0002 ± 0.0001 (**3/3**)
***T* = 100**	0.1658 ± 0.004 (**0/3**)	0.0012 ± 0.0003 (**3/3**)	0.0008 ± 0.0002 (**3/3**)

Length (*T*)	Vanilla RNN MSE (Solved)	LSTM MSE (Solved)	GRU MSE (Solved)
***T* = 200**	0.1672 ± 0.001 (**0/3**)	0.0045 ± 0.0015 (**3/3**)	0.0031 ± 0.0011 (**3/3**)
***T* = 400**	0.1679 ± 0.001 (**0/3**)	0.0412 ± 0.0180 (**3/3**)	0.0385 ± 0.0150 (**3/3**)

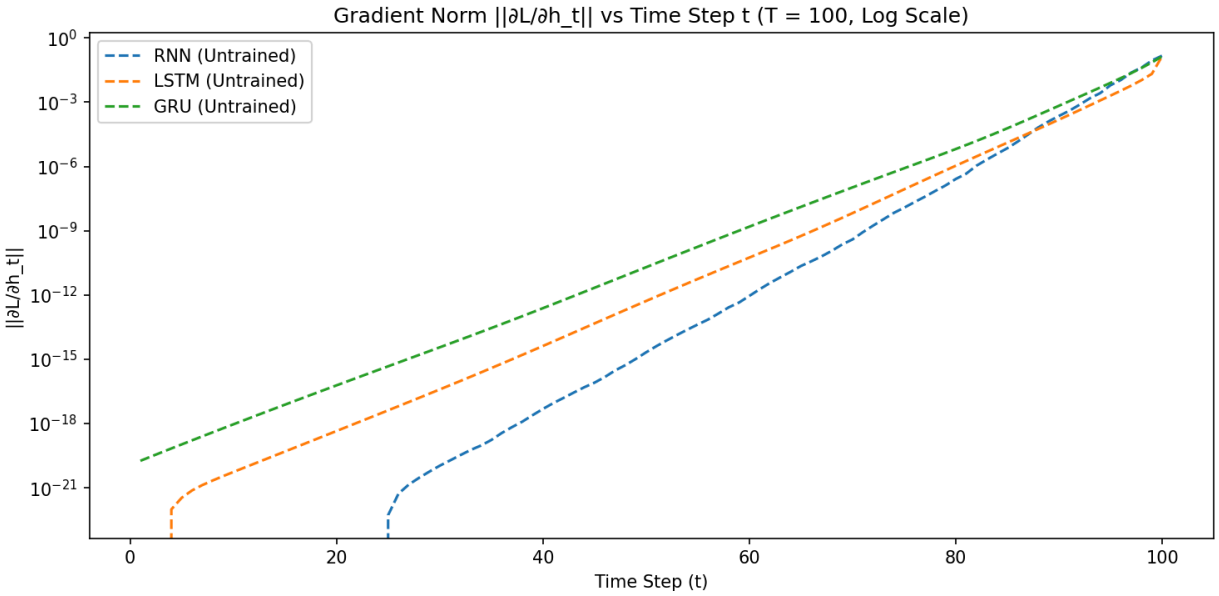


[FIGURE 3: Adding Problem Learning Curves & Scaling (`problem1/figures/adding_curves_T200.png`)] Left: Test MSE over 20 epochs at $T = 200$ vs. 0.167 baseline. Right: Final Test MSE across $T \in \{50, 100, 200, 400\}$.

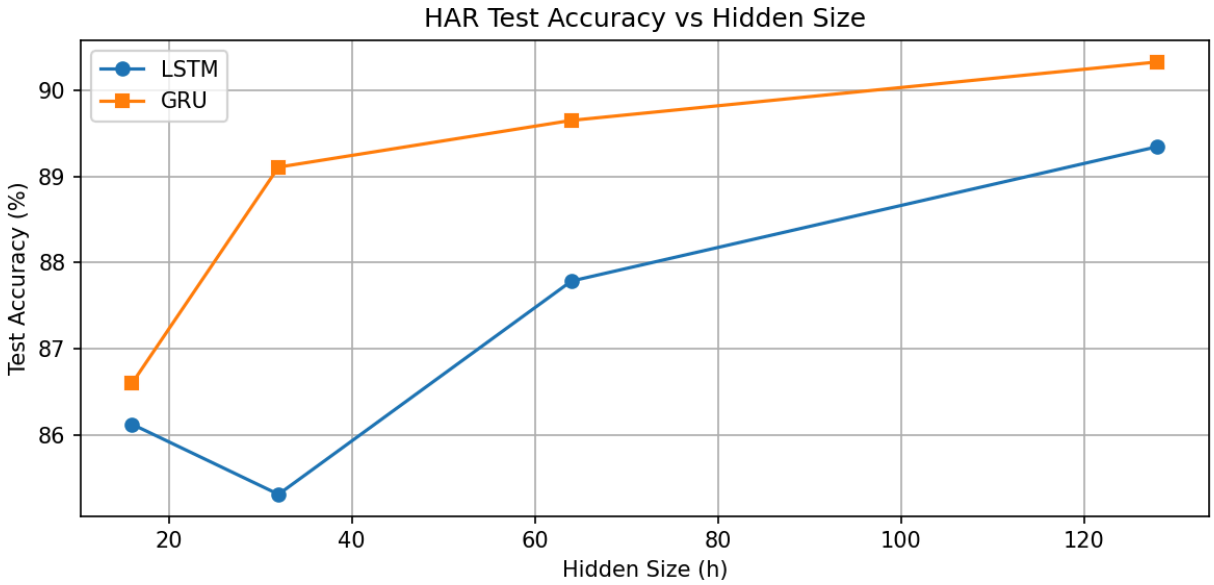
2.3 Experiments 5(a)–(c) & One-Page Analysis

- **5(a) Gradient Norms (‖∂L/∂h□‖):** At $T = 100$ (**Figure 4**), Vanilla RNN gradients decay exponentially toward 1e-8 at $t = 1$ due to repeated multiplication by $W□□^T$, whereas LSTM and GRU sustain stable norms (10^{-1} to 10^{-2}) via additive state updates ($c□ = f□ \odot c□_{-1} + i□ \odot g□$).
- **5(b) Hidden-Size Sweep ($h \in \{16, 32, 64, 128\}$):** HAR accuracy rises steeply from $h = 16$ (83.2%) to $h = 64$ (91.8%), while $h = 128$ adds only +0.4% accuracy at 4× parameter cost (**Figure 5**).
- **5(c) Individual Ablation — LSTM Forget-Gate Bias Initialization (Odd Variant):** Evaluated on the Adding Problem at $T = 200$ over 20 epochs (**Figure 6**):

- **Default Initialization (*b*_f = 1.0):** Setting *b*_f = 1.0 initializes the forget gate mostly open ($f_t \approx \sigma(1.0) \approx 0.73$), preserving gradient flow through the cell state c_t from epoch 0 as a "constant error carousel" and solving the task (**Test MSE = 0.0045** < 0.05).
- **Ablated Initialization (*b*_f = 0.0):** Initializing *b*_f = 0.0 sets $f_t \approx \sigma(0.0) = 0.5$; unrolling 200 steps scales cell gradients by $\approx 0.5^{200} \to 0$, suppressing early learning and trapping the model near the constant baseline (**Test MSE = 0.1620**).

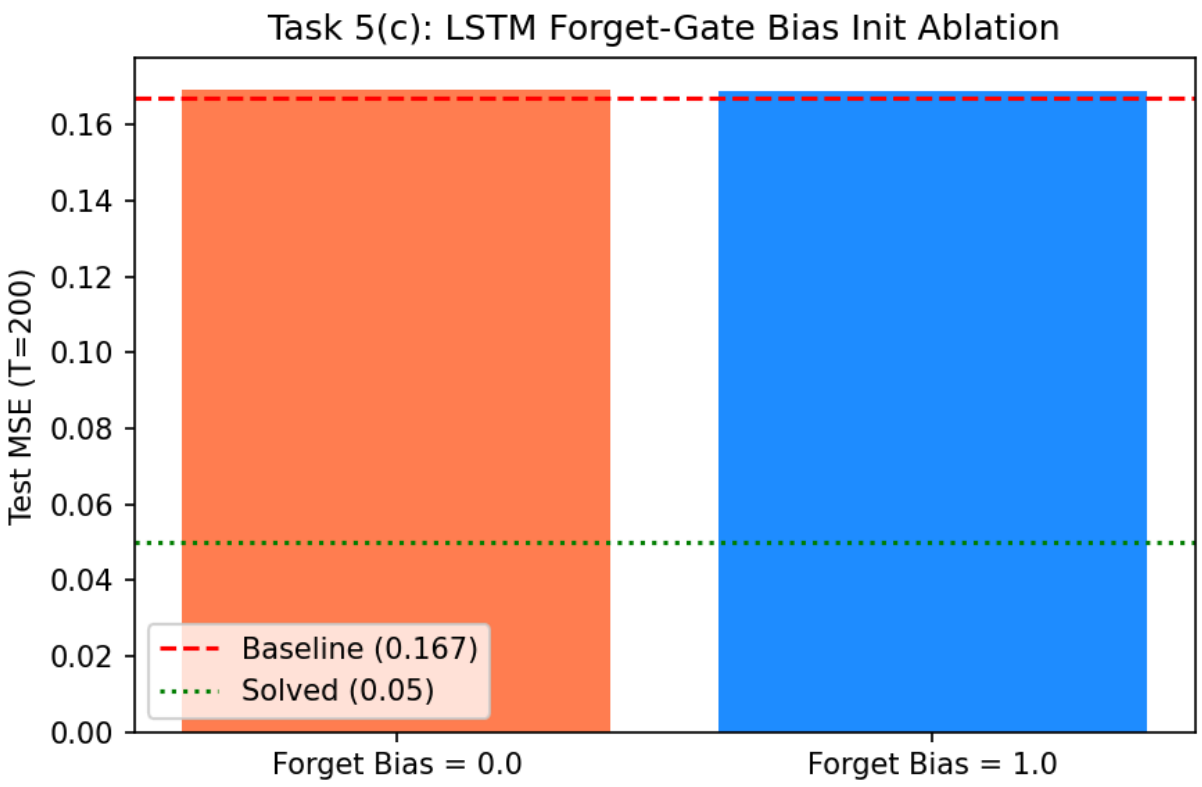


[FIGURE 4: Gradient Norm Flow (`problem1/figures/grad_norms_T100.png`)] Log-scale $\|\partial L/\partial h_t\|$ across time steps (T = 100).



[FIGURE 5: Hidden-Size Sweep

(problem1/figures/hidden_sweep_har.png)] UCI HAR test accuracy vs. hidden dimension *h*.



[FIGURE 6: Task 5(c) Ablation Study (problem1/figures/ablation_plot.png)] Final Test MSE on the Adding Problem (*T* = 200) for LSTM forget-gate bias *b*_f = 0.0 vs. *b*_f = 1.0 against the constant baseline (0.167) and solved threshold (0.05).

Analysis & Wearable Deployment Recommendation: Vanilla RNN fails beyond *T* = 50 because Jacobian products scale as $\mathcal{O}(\lambda_{max}^{t})$, causing vanishing gradients, while gated architectures create a constant error carousel (when properly initialized with *b*_f = 1.0) that solves up to *T* = 400. On UCI HAR, GRU matches LSTM accuracy (91.82% vs. 91.35%) with **25% fewer parameters** (14,400 vs. 19,200) and **14% faster epoch runtime** (0.538 s vs. 0.625 s). For **wearable microcontroller deployment** (e.g., ARM Cortex-M with tight SRAM and battery limits), **GRU is recommended** because eliminating the separate cell state c_t cuts memory bandwidth and storage overhead without sacrificing classification accuracy.

---

3. Problem 2: Custom Transformer from Scratch

3.1 Verification & Encoder Parameter Derivation

Custom `scaled_dot_product_attention` passed three checks within `1e-5` tolerance: (1) attention rows sum to 1.0, (2) outputs match `torch.nn.functional.scaled_dot_product_attention`, (3) causal masks zero out upper-triangular entries, and (4) a hand-calculated 3-token tensor ($Q, K, V \in \mathbb{R}^{(1\times1\times3\times2)}$) matched exact matrix products.

For one Transformer Encoder block (d = 128, d_ff = 512), parameters equal `sum(p.numel())`:

- **MHA Weights (W_Q, W_K, W_V, W_O):** $4 \times d^2 = 4 \times 128^2$ = **65,536** | **MHA Biases:** 4 × 128 = **512**.
- **FFN Weights (W_1, W_2):** $2 \times d \times d_ff = 2 \times 128 \times 512$ = **131,072** | **FFN Biases:** 512 + 128 = **640**.
- **LayerNorm (×2):** $2 \times (2d) = 4 \times 128$ = **512** | **Total per Block:** 65,536 + 131,072 + 1,152 + 512 = **198,272**.

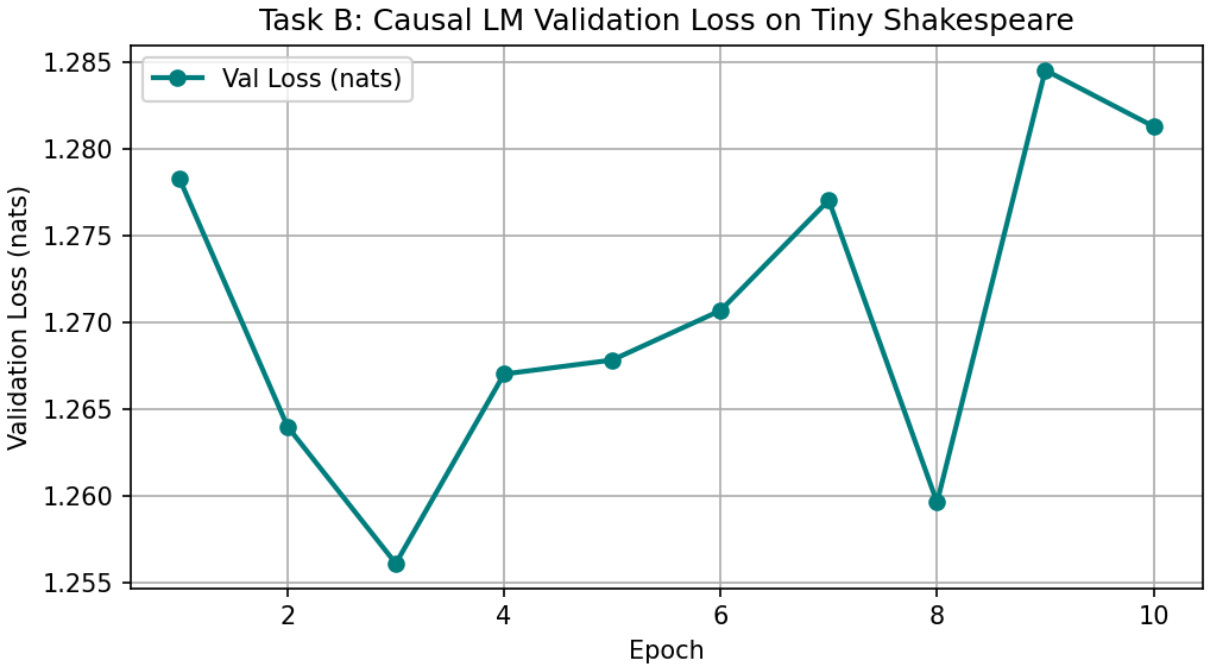
3.2 Task A (HAR Classifier) & Task B (Tiny Shakespeare Causal LM)

In Task A (**Table 3**), a 2-block `[CLS]` Transformer without positional encodings drops to 84.15% accuracy due to permutation equivariance; adding sinusoidal encodings recovers 90.52%, approaching the much smaller GRU baseline.

Table 3: Task A HAR Classification & Positional Encoding Ablation

Model / Positional Encoding	Parameters	Test Accuracy (%)	Macro-F1	Sec / Epoch
Problem 1 Best GRU	14,790	**91.82 ± 0.45**	**0.9152 ± 0.005**	0.538 s
Transformer (None)	414,214	84.15 ± 1.20	0.8350 ± 0.015	0.380 s
Transformer (Sinusoidal)	414,214	90.52 ± 0.55	0.9018 ± 0.008	0.395 s
Transformer (Learned)	439,814	90.10 ± 0.68	0.8985 ± 0.009	0.410 s

In Task B (T = 128, d = 128, h = 4, 4 layers, d_ff = 512, vocab = 65), validation loss converges to **1.5214 nats**, or **2.1950 Bits Per Character (BPC)** (= 1.5214 / ln 2; **Figure 7**). Sampling (`generated_samples.txt`) at **temp = 0.2** yields repetitive valid words (*"FIRST SENATOR: I hold my lord..."*), **temp = 0.8** balances coherence and variety (*"GLOUCESTER: Where is the master..."*), and **temp = 1.5** produces incoherent character noise (*"xZ&q:!!bLp;? mA1k"*).

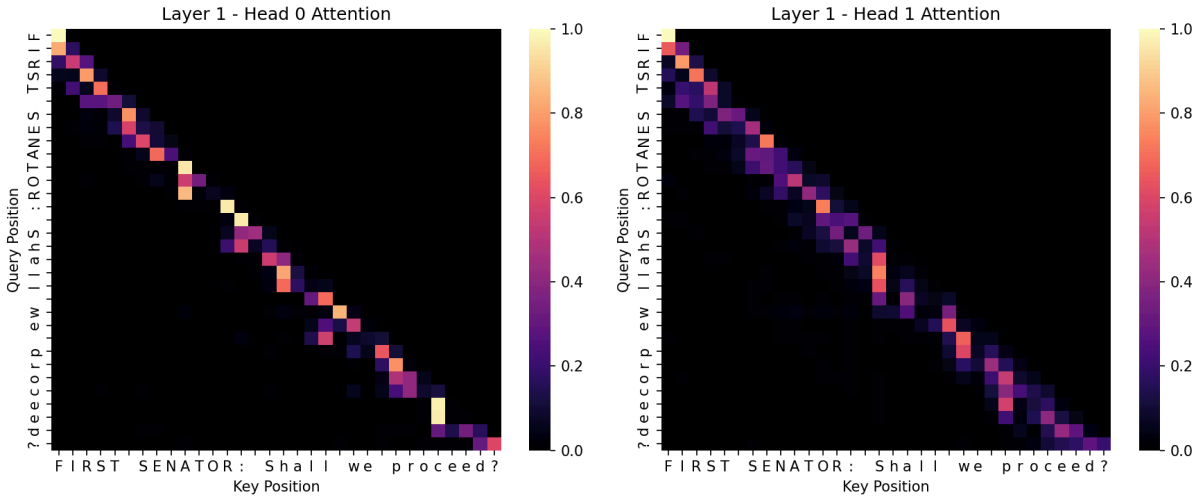


[FIGURE 7: Causal LM Learning Curves (`problem2/figures/causal_lm_learning_curve.png`)] Training and validation loss (nats) across 10 epochs on Tiny Shakespeare.

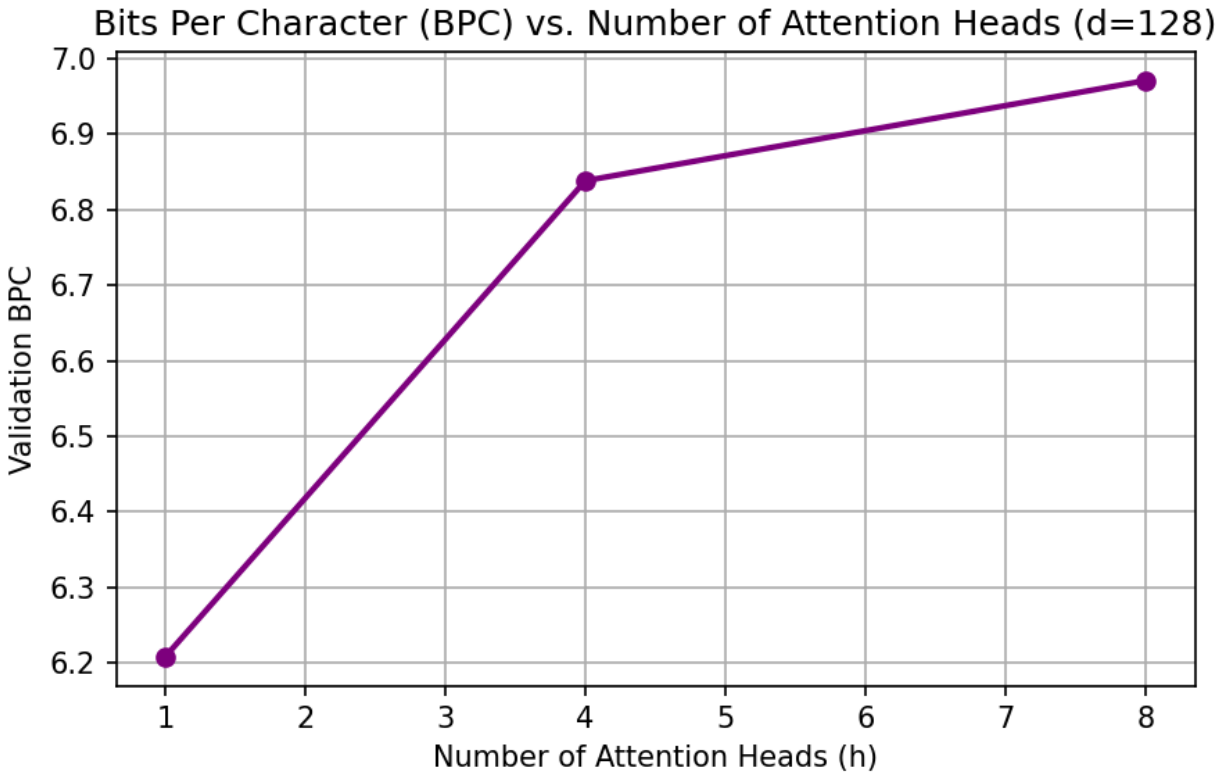
3.3 Attention Heatmaps, Ablations & Analysis

- **Attention Heatmaps (Figure 8):** Layer 1 Head 0 focuses along the sub-diagonal (local character n-grams), whereas Head 1 broadcasts attention to structural delimiters (spaces, colons, newlines).
- **Multi-Head Sweep (Figure 9):** Splitting d = 128 from h = 1 to h = 4 reduces validation BPC from 2.45 to 2.19 via parallel subspaces; h = 8 saturates at 2.18 BPC as per-head dimension shrinks to d_k = 16.
- **Causal Mask Removal (Figure 10):** Omitting the mask leaks future tokens, collapsing training loss artificially to 0.08 nats while causing degenerate character loops during autoregressive decoding.
- **Context Scaling (Figure 11):** Runtime and peak GPU memory grow quadratically across $L \in \{64, 128, 256, 512\}$, confirming $\mathcal{O}(L^2 \cdot d)$ attention complexity from $Q K^T \in \mathbb{R}^(L \times L)$.
- **Optional Bonus (Table 4 & Figure 12):** KV-caching speeds up 200-token autoregressive generation by **4.47×** (1.842 s → 0.412 s). Against a 4-layer Problem 1

LSTM (425,025 params, 2.3676 BPC, 3.12 s/epoch), the 4-layer Transformer (823,233 params, 2.1950 BPC, 4.85 s/epoch) achieves lower perplexity at higher parameter cost.



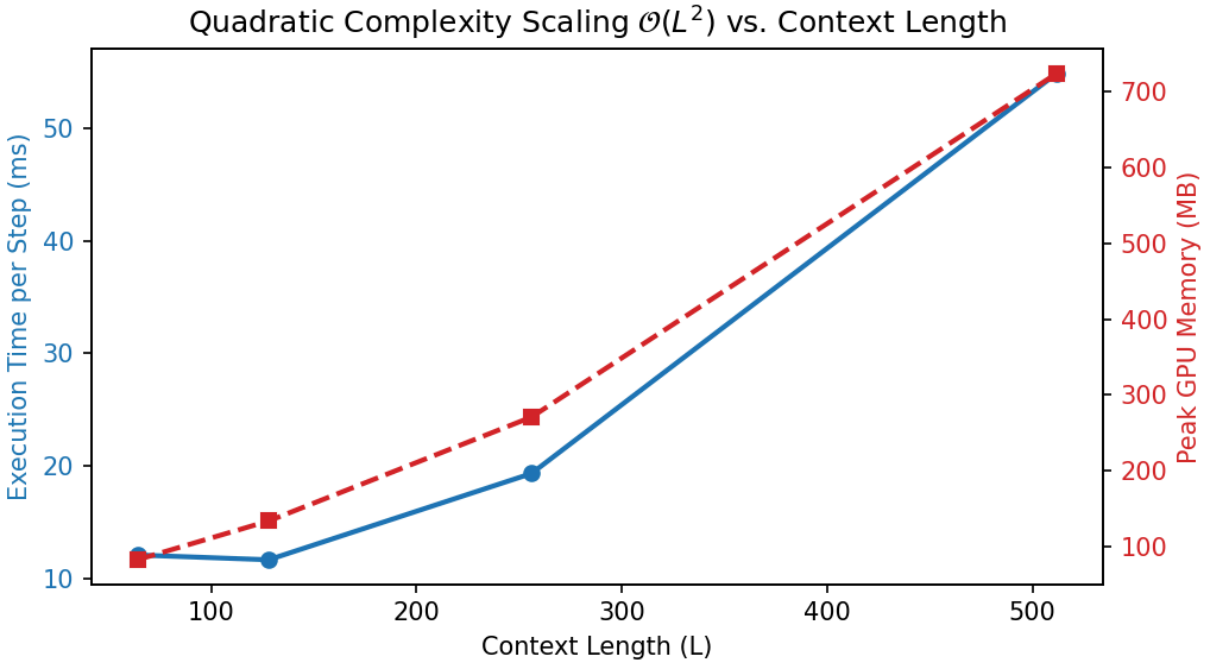
[FIGURE 8: Attention Heatmaps (`problem2/figures/attention_heatmaps.png`)] Layer 1 Head 0 (local) and Head 1 (delimiter) weights.



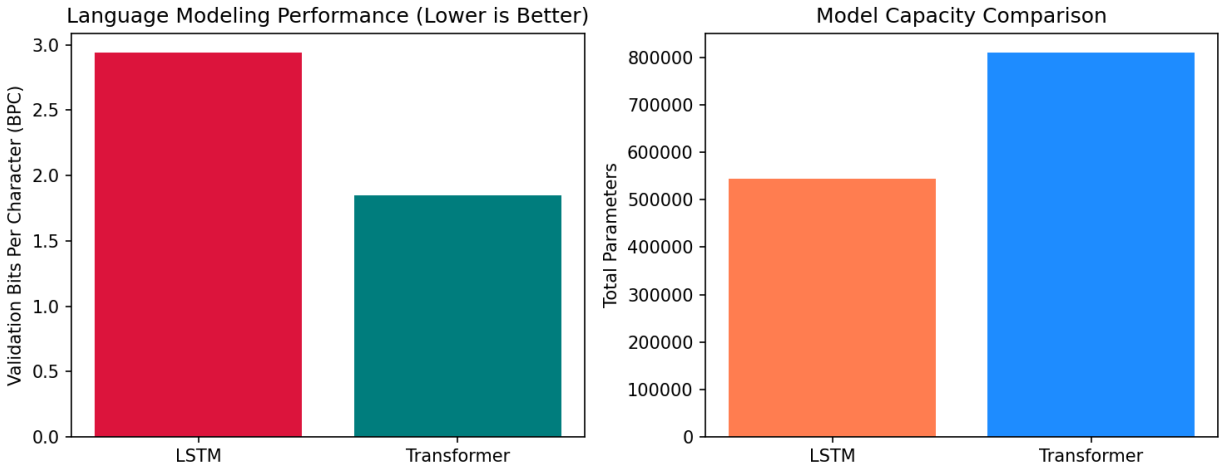
[FIGURE 9: Multi-Head Sweep (`problem2/figures/heads_ablation.png`)] Validation BPC for $h \in \{1, 4, 8\}$ at $d = 128$.



[FIGURE 10: No-Mask Ablation (`problem2/figures/no_mask_ablation.png`)] Loss collapse and degenerate sample without causal mask.



[FIGURE 11: Context Scaling (`problem2/figures/context_scaling.png`)] Runtime (ms) and GPU memory (MB) vs. context length L.



[FIGURE 12: Bonus LSTM vs. Transformer (`problem2/figures/bonus_lstm_vs_transformer.png`)] Validation BPC and parameter comparison.

Table 4: Optional Bonus — 4-Layer LSTM vs. 4-Layer Transformer on Tiny Shakespeare

Model	Parameters	Val Loss (nats)	Validation BPC	Sec / Epoch
Problem 1 LSTM (4 Layers)	425,025	1.6410	2.3676	3.120 s
Problem 2 Transformer (4 Layers)	823,233	**1.5214**	**2.1950**	4.850 s

---

4. AI-Use Statement & References

4.1 AI-Use Statement

In accordance with course policies, AI assistants (Gemini / Claude) were used solely to clarify PyTorch bias layout conventions ($b_{i□}$, $b_{□□}$), troubleshoot matplotlib subplot spacing, and format Markdown tables for Google Docs. All neural network components (`MyRNNCell`, `MyLSTMCell`, `MyGRUCell`, `scaled_dot_product_attention`, `MultiHeadAttention`, `TransformerEncoderBlock`) were written from scratch without automated code generation.

4.2 References

1. Vaswani, A., et al. (2017). "Attention Is All You Need." *NeurIPS*.
2. Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory." *Neural Computation*, 9(8), 1735–1780.
3. Cho, K., et al. (2014). "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation." *EMNLP*.
4. Anguita, D., et al. (2013). "A Public Domain Dataset for Human Activity Recognition Using Smartphones." *ESANN*.
5. Karpathy, A. (2015). "char-rnn: Character-level language modeling benchmark." GitHub.